Contents

1 Task 1

1.1 Reading data from files

For the most part, in this course, we will either get data from other packages in R, or read data from files. It is important to be familiar with how to import data, and generally there are only two (three) functions to be aware of, `read.fwf()` and `read.table()` (and `read.csv()`, which is really just `read.table()` with some preset settings).

In general, the first step is to be aware what is the structure of the data. Open the data file you want to import and study its structure. Does it have a header? Are the individual data points in an observation separated by anything? And in general, most files (almost all) should have one observation in one row.

1.2 The working directory (`getwd()` and `setwd()`)

If you encounter any file-related errors (e.g. file not found) when trying to read a file, the first thing you should check is your working directory. Most problems will likely be solved if you check your working directory, because **any functions that require a file (filepath) will try to find the file (filepath) relative to your current working directory.**

To see your current working directory, use `getwd()`. You may, for example, get the following response: `C:/Users/caleb/Documents`. Thus, if I run `read.table("lab1fixed.txt")`, it will look for the file at `C:/Users/caleb/Documents/lab1fixed.txt`. So, check if the file is where you intend it to be, or check that your working directory is where you intend it to be.

To change your working directory, you can use `setwd()` and pass it a path.

Alternatively, in RStudio, use the Files tab, click the 3 dots at the right of the panel and navigate to the folder with your files. Click the cog icon, and find the 'Set As Working Directory' option.

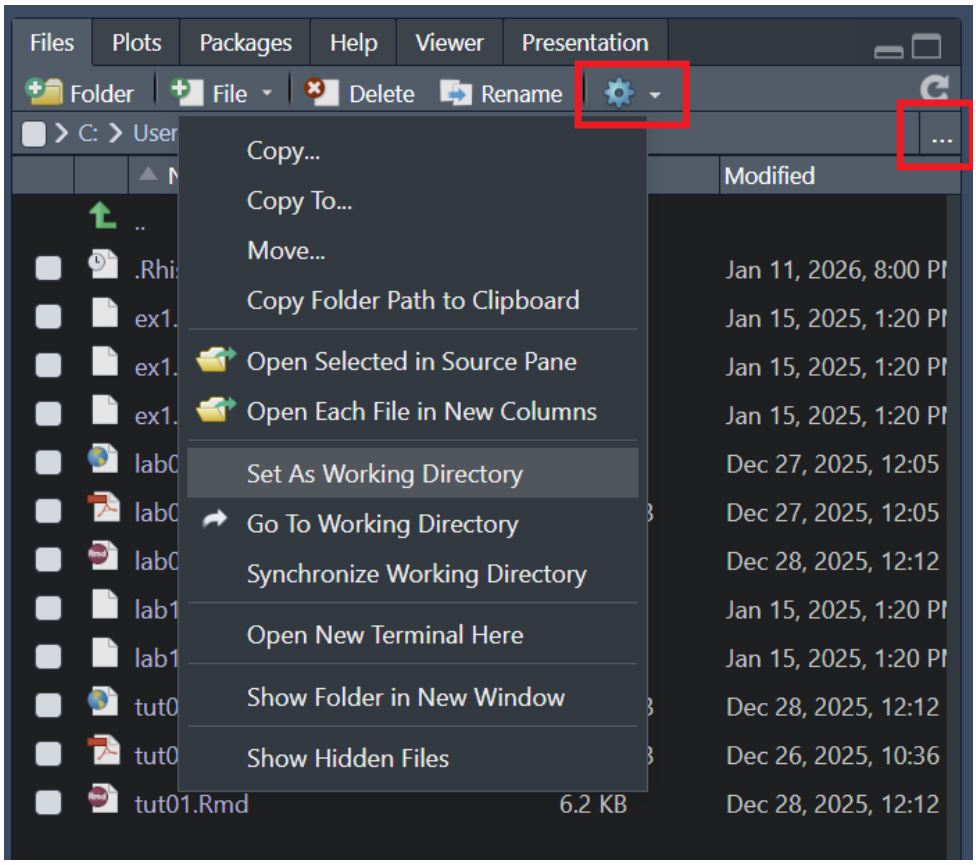
Both methods are equivalent, as they ultimately use the `setwd()` function - it will appear in the R console.

A quick mention for Rmarkdown (`.Rmd`) files: I find that as long as the files are in the same folder (e.g. `Documents/script.Rmd` and `Documents/data.csv`) the working directory is irrelevant.

1.3 Importing fixed width format files `read.fwf()`

Open and observe the format of `lab1fixed.txt`. As the name suggests, it is a **fixed-width format** file, with no separators between each data point in a single observation. Instead, some number of columns corresponds to some information about a single observation, with the details for this file provided in the lab sheet.

To import this file into R, use the `read.fwf()` command. To understand what the function does and what parameters it can take, we can use the `?` operator, i.e. `?read.fwf`. Read through the Description,



Usage, and Arguments sections to understand how to use the function. Most help pages may also provide some example usage at the bottom of the page.

```
lab1 <- read.fwf('lab1fixed.txt', widths = c(3, 1, 3, 2, 1),
   col.names = c("id", "gender", "height", "weight", "siblings"))
```

1.4 Filtering entries

To filter the entries, we can use a Boolean test with indexing. As a mini example, to demonstrate what occurs when filtering:

```
df1 <- data.frame(matrix(1:10, nrow=5))
df1
```

```
## # A tibble: 5 x 2
##      X1    X2
##   <int> <int>
## 1     1     6
## 2     2     7
## 3     3     8
## 4     4     9
## 5     5    10
```

```
isodd <- df1$X1 %% 2 == 1   # %% is modulo operator, == is equality test
↪  operator.
isodd         # vector of T and F values, depending on whether X1 was odd
↪  or not.
```

```
## [1]  TRUE FALSE  TRUE FALSE  TRUE
```

```
df1[isodd, ]  # select only the entries which correspond to T in isodd,
↪  i.e. the odd entries.
```

```
## # A tibble: 3 x 2
##      X1    X2
##   <int> <int>
## 1     1     6
## 2     3     8
## 3     5    10
```

We can combine defining the filter criteria and applying it into one line.

```
lab1m <- lab1[lab1$gender == "M", ]
dim(lab1m) # and look at the first entry (rows)
```

```
## [1] 48  5
```

```
nrow(lab1m) # returns only the number of rows
```

```
## [1] 48
```

And thus we conclude that there are 48 males in the data frame `lab1`.

1.5 Importing other data files (`read.table()`)

Now, look at `lab1test.txt` and observe its format. There are two columns, with entries separated by a space character. The first row are the headers of the table.

The most general function available to us is `read.table()`. Again, look at the documentation of it using `?read.table`. There are many options to it, but the most important would be the `header` and `sep` parameters.

```
lab1test <- read.table('lab1test.txt', header=T)
```

Try experimenting with setting `header=F` and see what it does to the resulting data frame.

1.6 Merging data frames

The `lab1test` data frame has column headers `id` and `test`. The `id` column in `lab1test` corresponds to the `id` column in `lab1` and extends the data. Thus, we want to merge the data frames based on the `id` column, which we can do using `merge()`. Again, you can use `?merge` to understand its usage. In this case, it is a simple use of the function, but we can specify on what columns it merges on with the `by` parameter.

```
lab1merge <- merge(lab1, lab1test)
dim(lab1merge)
```

```
## [1] 120   6
```

To complete the rest of part (c), it is another filtering exercise, this time on the heights.

```
lab1merge[lab1merge$height > 182, 'test']
```

```
## [1] 55 76 54
```

From which we can see that there are only three individuals with height greater than 182.

1.7 Removing a row from a data frame

Removing a row is similar to filtering, and we must understand what criteria we want to filter or remove on. Since we want to remove the record related to subject 211, i.e. with `id` 211, we can filter using the `id` column, taking everything *but ID 211*.

```
lab1remo <- lab1merge[lab1merge$id != 211,]
dim(lab1remo)
```

```
## [1] 119   6
```

From which, comparing with the dimensions of `lab1merge`, we can see that one entry was indeed removed.

1.8 Modifying a data frame entry

Modifying a data frame entry is mostly a job of indexing and filtering, then assigning the new data. Similar to the above, we want to change the weight of `id` 211 from 60 to 80, so:

```
lab1[lab1$id == 211,]
```

```
## # A tibble: 1 x 5
##      id gender height weight siblings
##   <int> <chr>   <int>  <int>    <int>
## 1   211 M         171     60        3
```

```
lab1[lab1$id == 211, 'weight'] = 80
lab1[lab1$id == 211,]
```

```
## # A tibble: 1 x 5
##      id gender height weight siblings
##   <int> <chr>   <int>  <dbl>    <int>
## 1   211 M         171     80        3
```

1.9 Sorting a data frame (`order()`)

To sort a data frame, we can use the `order()` function, which just returns a numerical ranking to the data. For example:

```
x <- c(100, 200, 150, 175, 125)
x[c(3,3,2,4,4)] # get the 3rd, 3rd, 2nd, 4th, 4th entries.
```

```
## [1] 150 150 200 175 175
```

```
order(x) # 1, 5, 3, 4, 2
```

```
## [1] 1 5 3 4 2
```

```
x <- x[order(x)]
x
```

```
## [1] 100 125 150 175 200
```

In earlier exercises, we used a boolean condition (true or false, based on some criteria) to determine whether to keep or discard a row (observation). We can also use a numerical index to determine what entries to take, as in line 2 in the above code block.

You can see that `order()` gives the ordering of the numbers from smallest (1) to largest (5). So, when we order x by this sequence of indexes, we will get the elements in x ordered from smallest to largest.

In part (f), we are also asked to filter the data. In this case, it does not matter in which order we do it (filter then order, or order then filter). To simplify the ordering, we will use the `decreasing` parameter in `order()` to get it to order it in descending order, i.e. largest to smallest. `order()` also only can sort **vectors, and not dataframes/matrices**. If you experience an error when using `order()`, check that it is **not a dataframe/matrix**.

```
lab1f <- lab1merge[lab1merge$gender == 'F',]    # filter the data
ordering <- order(lab1f$height, decreasing=T)   # get the height order
↪  vector
lab1ordered <- lab1f[ordering,]
lab1ordered[2, c('height', 'weight', 'test')]   # get the second entry and
↪  relevant columns
```

```
## # A tibble: 1 x 3
##   height weight  test
##    <int>  <int> <int>
## 1    174     64    57
```

2 Task 2

We will define the matrices X and Y first, using `rep()` and `seq()` for X.

`rep()` repeats what is passed to it, and `seq()` functions essentially like the `range()` function in Python, but is inclusive of the end number. Again, to read more about it, use the `?` operator on them.

```
X <- cbind(rep(1, 4), seq(1,7,2)) # rep() repeats `1` 4 times, seq() is
↪  similar to range() in Python
Y <- cbind(c(4, 6, 13, 20)) # make it a matrix! without cbind, Y will just
↪  be a vector.
```

$\hat{\beta}$ can then be calculated easily.

```
bhat <- solve((t(X) %*% X)) %*% t(X) %*% Y
bhat
```

```
##       [,1]
## [1,] -0.25
## [2,]  2.75
```

3 Task 3

3.1 Defining functions

Functions in R are defined with the `function` keyword and are assigned to variable names.

```
moment <- function(x, power=1) {
  if (power == 1) { return(mean(x)) }
  else { return(mean( (x - mean(x))**power )) }
}

moment(lab1$height)
```

```
## [1] 165.8667
```

```
moment(lab1$height,2)
```

```
## [1] 75.98222
```

4 Task 4

This is the function defined on slide 55. By default, functions in R return the last line run in the function, if it does not encounter a `return` statement first. You may also choose to explicitly return and end the function with a `return()` call.

```
f <- function(x) { return(-2*x**2 - 5*x + 7) }
```

Implementing the bisection algorithm:

```
root_bisection_algorithm <- function(f, xmin, xmax) {
  if (xmin > xmax) (return("xmin > xmax! please fix!"))

  xmid <- (xmin + xmax) / 2
  cat("xmin =", xmin, "xmax =", xmax, "\n")

  while (abs(f(xmid)) > 10**-3 & abs(xmin - xmax) > 10**-3 ) {
    if (f(xmin) * f(xmid) < 0) { xmax = xmid }
    else { xmin = xmid }
    xmid <- (xmin + xmax) / 2

    cat("xmin =", xmin, "xmax =", xmax, "\n")
  }
```

```
  if (abs(f(xmid)) > 10**-3) { return("no root found in interval!")}
  xmid
}

root_bisection_algorithm(f, -4, 0)
```

```
## xmin = -4 xmax = 0
## xmin = -4 xmax = -2
## xmin = -4 xmax = -3

## [1] -3.5
```

```
root_bisection_algorithm(f, -5, 0)
```

```
## xmin = -5 xmax = 0
## xmin = -5 xmax = -2.5
## xmin = -3.75 xmax = -2.5
## xmin = -3.75 xmax = -3.125
## xmin = -3.75 xmax = -3.4375
## xmin = -3.59375 xmax = -3.4375
## xmin = -3.515625 xmax = -3.4375
## xmin = -3.515625 xmax = -3.476562
## xmin = -3.515625 xmax = -3.496094
## xmin = -3.505859 xmax = -3.496094
## xmin = -3.500977 xmax = -3.496094
## xmin = -3.500977 xmax = -3.498535
## xmin = -3.500977 xmax = -3.499756
## xmin = -3.500366 xmax = -3.499756

## [1] -3.500061
```

```
root_bisection_algorithm(f, -10, -9)
```

```
## xmin = -10 xmax = -9
## xmin = -9.5 xmax = -9
## xmin = -9.25 xmax = -9
## xmin = -9.125 xmax = -9
## xmin = -9.0625 xmax = -9
## xmin = -9.03125 xmax = -9
## xmin = -9.015625 xmax = -9
## xmin = -9.007812 xmax = -9
```

```
## xmin = -9.003906 xmax = -9
## xmin = -9.001953 xmax = -9
## xmin = -9.000977 xmax = -9

## [1] "no root found in interval!"
```